

クラシックエディターを実践的にカスタマイズする

Studioから新しいサイトを立ち上げよう「my-second-test」など。

ベーステーマをインストールしよう

外観のテーマから「Twenty

Twelve」をインストールしよう(だめだったら公式のzipをインストール)。

ベーステーマは2012年と古いが、メンテナンスはされており、余計なコードが少なくベースとしては活用しやすい。

functionsにはいろいろな処理が書いているので、さらっと読んでみよう。

テーマのカスタマイズ内容

画面要素	表示内容	対応コード・関数 例	対応ファイル
ヘッダー	サイトタイトル「Twenty Twelve」、サブタイトル、水平ナビゲーション	<code>wp_nav_menu()</code> によるメニュー表示	<code>header.php</code>
ナビゲーションバー	黒背景に白文字のメニュー項目 + 検索アイコン	<code>register_nav_menus()</code> → <code>wp_nav_menu()</code> + CSS調整	<code>functions.php</code> , <code>header.php</code> , <code>style.css</code>
投稿一覧 (2件)	サムネイル画像、タイトル、投稿日、カテゴリ、抜粋、"Read more" リンク	<code>the_post_thumbnail()</code> , <code>the_title()</code> , <code>get_the_date()</code> , <code>the_category()</code> , <code>the_excerpt()</code>	<code>index.php</code> , <code>content.php</code>
サイドバー	「RECENT POSTS」見出しとリンク	<code>dynamic_sidebar()</code> によるウィジェット表示	<code>sidebar.php</code> , <code>functions.php</code>
背景とレイアウト	メイン白背景、サイドバー灰色、全体に余白と区切り線	<code>.post-grid</code> , <code>.post-card</code> , <code>.sidebar</code>	<code>style.css</code>

		などのCSSクラス	
--	--	-----------	--

子テーマを用意しよう

テーマを直接書き換えた場合に、アップデートが入ると上書きされて消えてしまうという問題が発生する。

その問題を回避するため、子テーマを用意して親テーマを継承し、必要な部分をカスタマイズするという形式を取る。

子テーマの作成

WordPressの /wp-content/themes/ に twentytwelve-child というフォルダを作成

twentytwelve-child/style.cssの作成

```
/*
Theme Name: Twenty Twelve Child
Template: twentytwelve
Version: 1.0
*/

body {
    background-color: #f9f9f9;
}
```

twentytwelve-child/functions.php の作成 (CSSの継承)

親テーマのCSSを読み込んだ上で、子テーマのCSSを追加

```
<?php
function twentytwelve_child_enqueue_styles() {
    wp_enqueue_style( 'twentytwelve-style', get_template_directory_uri() . '/style.css' );
    wp_enqueue_style( 'twentytwelve-child-style', get_stylesheet_uri(),
        array( 'twentytwelve-style' ) );
}
add_action( 'wp_enqueue_scripts', 'twentytwelve_child_enqueue_styles' );
```

子テーマの有効化

管理画面から子テーマの「twentytwelve-child」を有効化

パーマリンクをカスタムの「カテゴリー名/投稿名」に変更

Step 1 : 投稿一覧ページのテンプレート作成 (index.php)

目的 : 投稿が並ぶページの構造を理解し、タイトル・本文・画像を表示する

ファイル : twentytwelve-child/index.php

- WordPressが投稿を自動で並べてくれる仕組みを体験
- 投稿のタイトル・本文・画像を表示するHTML構造を作成
- 「投稿があるかどうかを確認して、1つずつ表示する」流れを理解

関数の補足

- get_header() や get_footer() は「header.php, footer.phpなどの共通部分を読み込む命令」
- the_title() や the_content() は「title, 本文などの投稿の中身を表示する命令」

```
<?php get_header(); ?>

<div class="main-area">
  <div class="post-grid">
    <?php if ( have_posts() ) : while ( have_posts() ) : the_post(); ?>
      <article class="post-card">
        <a href="<?php the_permalink(); ?>">
          <?php if ( has_post_thumbnail() ) : ?>
            <div class="thumb"><?php the_post_thumbnail('medium'); ?></div>
          <?php endif; ?>
          <h2><?php the_title(); ?></h2>
        </a>
        <div class="meta">
          <span class="date"><?php echo get_the_date(); ?></span>
          <span class="cat"><?php the_category(', '); ?></span>
        </div>
        <div class="excerpt"><?php the_excerpt(); ?></div>
      </article>
    <?php endwhile; endif; ?>
  </div>

  <?php get_sidebar(); ?>
</div>

<?php get_footer(); ?>
```

Step 2 : 投稿詳細ページ (single.php)

目的

- 投稿1件の詳細ページの構造を理解し、デザインに合わせてカスタマイズする

- Before (親テーマ) と After (子テーマ) の違いをコードと画面で把握する

```
<?php get_header(); ?>

<article class="single-post">
  <h1><?php the_title(); ?></h1>
  <div class="meta">
    <span class="date"><?php echo get_the_date(); ?></span> /
    <span class="author"><?php the_author(); ?></span>
  </div>
  <?php if ( has_post_thumbnail() ) : ?>
    <div class="thumb"><?php the_post_thumbnail('large'); ?></div>
  <?php endif; ?>
  <div class="content"><?php the_content(); ?></div>
</article>

<?php get_sidebar(); ?>
<?php get_footer(); ?>
```

Step 3 : 固定ページ (page.php)

```
<?php get_header(); ?>

<main class="page-content">
  <h1><?php the_title(); ?></h1>
  <div class="content"><?php the_content(); ?></div>
</main>

<?php get_footer(); ?>
```

Step お問い合わせページの作成手順(固定ページ)

お問い合わせ用のテンプレートを作成する(page-contact.php)。

```
<?php
/*
Template Name: Contact Page
*/
get_header(); ?>

<main class="contact-page">
  <h1><?php the_title(); ?></h1>

  <div class="content">
    <?php
      // 固定ページの本文を表示 ( ショートコードも含まれる )
      the_content();
    ?>
  </div>
</main>

<?php get_footer(); ?>
```

お問い合わせプラグインを入れよう

- 管理画面から、「Contact Form 7」を導入する
- インストールから有効化まで実行する

サイドメニューからデフォルトのフォームを確認し、以下のような項目を生成する(GUI)

メール2を使用し、相手先への自動返信メールを作成する。

警告が出るが、プラグイン元のページでスパム対策についての注意を喚起される。

```
題名: お問い合わせありがとうございます | [_site_title] "[your-subject]"
```

固定ページにお問い合わせフォームを設置しよう

- 固定ページから「お問い合わせ」というページを作成しよう
- テンプレートをお問い合わせように設定しよう
- お問い合わせのショートコードをブロックでろう

```
[contact-form-7 id="{{ここにID}}" title="{{ここにタイトル}}"]
```

ローカル環境で実行はできないが、公開してページの動作までは確認しよう。

スタイルも調整しておこう(style.css。

```
.contact-page {
  max-width: 800px;
  margin: 0 auto;
  padding: 2rem;
  background: #fff;
}
.contact-page input,
.contact-page textarea {
  width: 100%;
  padding: 0.5rem;
  margin-bottom: 1rem;
  border: 1px solid #ccc;
}
.contact-page input[type="submit"] {
  background: #333;
  color: #fff;
  padding: 0.75rem 1.5rem;
  border: none;
  cursor: pointer;
}
```